

NextPermutation Dry Run

Dry Run of nextPermutation

Let's dry-run your **exact code** on:

```
nums = {2, 1, 5, 4, 3, 0, 0}
```

The easiest way to understand `nextPermutation` is to remember that it has **3 steps**:

1. Find the **pivot** (`ind`).
2. Find the smallest number on the right that is **greater than the pivot**, then swap.
3. Reverse the part after the pivot.

1. What does "next permutation" mean?

Suppose:

```
1 2 3
```

All permutations in sorted order are:

```
123
132
213
231
312
321
```

The **next permutation** after `213` is:

```
231
```

We want the permutation that is **just slightly larger**, not any random larger permutation.

2. Your array

```
2 1 5 4 3 0 0
```

Let's put indexes under it:

```
index:  0  1  2  3  4  5  6
        -----
nums:   2  1  5  4  3  0  0
```

Your code starts:

```
int n = nums.size();
int ind = -1;
```

So:

```
n = 7
ind = -1
```

STEP 1 – Find the pivot

Your loop:

```
for(int i = n-2; i >= 0; i--) {
    if(nums[i] < nums[i+1]) {
        ind = i;
        break;
    }
}
```

What are we looking for?

We're looking for the **first position from the right where the number increases**:

```
nums[i] < nums[i+1]
```

Why from the right?

Because the part on the right should be modified as little as possible.

Start

```
i = 5
```

Compare:

```
nums[5] = 0
nums[6] = 0
```

Check:

```
0 < 0
```

False.

So continue.

i = 4

```
nums[4] = 3
nums[5] = 0
```

Check:

```
3 < 0
```

False.

i = 3

```
nums[3] = 4
nums[4] = 3
```

Check:

```
4 < 3
```

False.

i = 2

```
nums[2] = 5
nums[3] = 4
```

Check:

```
5 < 4
```

False.

i = 1

```
nums[1] = 1
nums[2] = 5
```

Check:

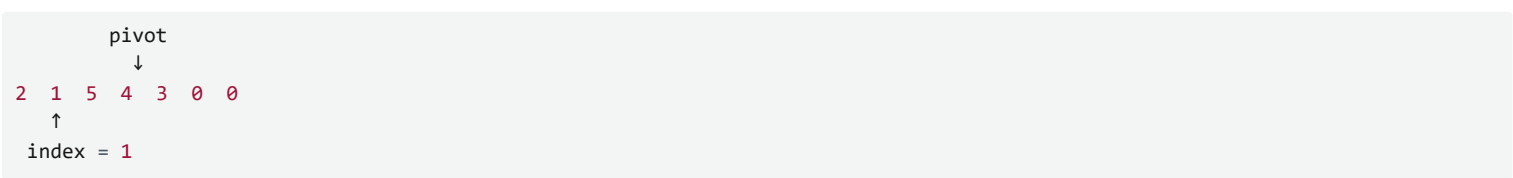
```
1 < 5
```

TRUE!

Therefore:

```
ind = 1;
break;
```

So:



The pivot is 1.

Why is 1 important?

Look at the suffix:

5 4 3 0 0

It is in **descending order**.

5 > 4 > 3 > 0 >= 0

This means this suffix is already the **largest possible arrangement** of those numbers.

So to make the entire number larger, we need to modify something before this suffix.

That's why we found:

1

STEP 2 – Find something greater than pivot

Now your code:

```
for(int i = n-1; i > ind; i--) {
    if(nums[i] > nums[ind]) {
        swap(nums[i], nums[ind]);
        break;
    }
}
```

Pivot:

nums[ind] = nums[1] = 1

We scan from the **right**.

Array:

2 1 5 4 3 0 0
 ↑
 pivot

i = 6

nums[6] = 0

Check:

0 > 1

False.

i = 5

nums[5] = 0

Check:

0 > 1

False.

i = 4

nums[4] = 3

Check:

3 > 1

TRUE.

So:

```
swap(nums[4], nums[1]);
```

Before:

```
2  1  5  4  3  0  0
  ↑      ↑
  1      3
```

After:

```
2  3  5  4  1  0  0
  ↑      ↑
```

So now:

```
2 3 5 4 1 0 0
```

Why did we choose 3?

This is important.

Pivot is:

```
1
```

Numbers to the right that are greater than 1:

```
5, 4, 3
```

The smallest is:

```
3
```

And because the suffix is descending, scanning from the right finds this smallest greater element.

So:

```
1 → 3
```

is the smallest increase we can make.

STEP 3 – Reverse the suffix

Now:

```
2 3 5 4 1 0 0
```

The pivot is at index 1.

Everything **after** the pivot:

```
5 4 1 0 0
```

is still in descending order.

Your code:

```
reverse(nums.begin() + ind + 1, nums.end());
```

Since:

```
ind = 1
```

we get:

```
reverse(nums.begin() + 2, nums.end());
```

So reverse:

```
5 4 1 0 0
```

into:

```
0 0 1 4 5
```

Final result:

```
2 3 0 0 1 4 5
```

Therefore:

```
[2, 3, 0, 0, 1, 4, 5]
```

Full dry run

Starting:

```
2 1 5 4 3 0 0
```

Find pivot

```
i=5 → 0 < 0 ✖  
i=4 → 3 < 0 ✖  
i=3 → 4 < 3 ✖  
i=2 → 5 < 4 ✖  
i=1 → 1 < 5 ✔
```

Therefore:

```
ind = 1
```

Find element greater than pivot

Pivot:

```
1
```

From right:

```
0 > 1 ✖  
0 > 1 ✖  
3 > 1 ✔
```

Swap:

```
2 1 5 4 3 0 0  
  ↓      ↓  
  1      3  
  
    ↓ swap  
2 3 5 4 1 0 0
```

Reverse suffix

```
2 3 | 5 4 1 0 0  
    ↓ ↓ ↓ ↓ ↓  
    reverse  
2 3 | 0 0 1 4 5
```

Final:

```
2 3 0 0 1 4 5
```

The intuition you should remember

The algorithm is basically saying:

Step 1

Find where I can make the number slightly bigger.

```
2 1 | 5 4 3 0 0  
  ↑  
  pivot
```

The suffix is already maximized, so we need to change 1.

Step 2

Replace 1 with the smallest number greater than 1.

1 → 3

giving:

2 3 5 4 1 0 0

Step 3

Now make everything after 3 as small as possible.

The suffix:

5 4 1 0 0

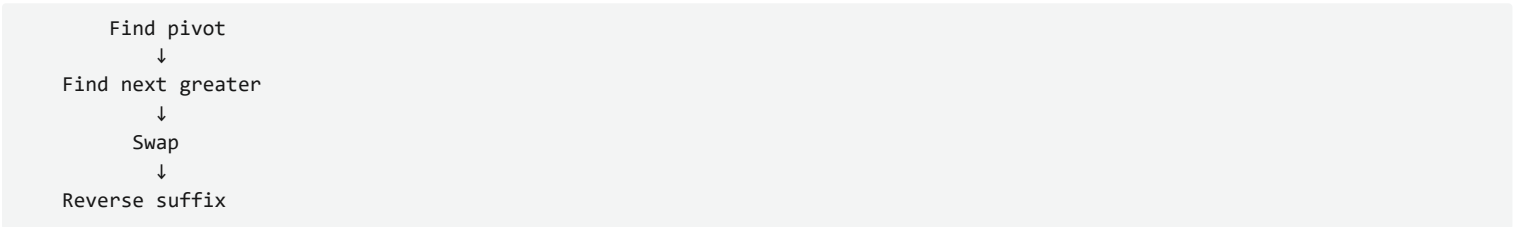
is descending, so reverse it:

0 0 1 4 5

Result:

2 3 0 0 1 4 5

That's why the algorithm is:



One subtle point

This line:

```
reverse(nums.begin() + ind + 1, nums.end());
```

does **not** reverse from ind.

It starts at:

ind + 1

because the pivot itself has already been correctly placed.

So if:

ind = 1

we reverse indexes:

2 → 6

not:

1 → 6

That distinction is important.